

当AI开始替你做事

Agent 时代的机会，风险与架构选择

主讲人：郑予彬

资深开发者布道师 | 亚马逊云科技

我们开始把AI当成“可以协作的对象”

越来越多的企业正在把AI接入业务流程

《Four Futures for Jobs in the New Economy: AI and Talent in 2030》 Published on 7 January 2026

2022 → 55%企业使用AI

2025 → 88%企业使用AI

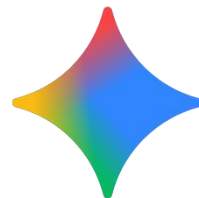
AI正从“回答问题”走向“参与任务”

《Which Economic Tasks are Performed with AI?》

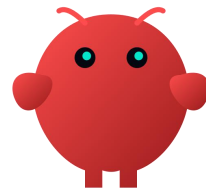
Anthropic 分析了数百万次 Claude 对话，发现一个很有意思的结果，AI 使用模式为：

能力增强(Augmentation) 57%

任务自动化(Automation) 43%



我的AI同事们

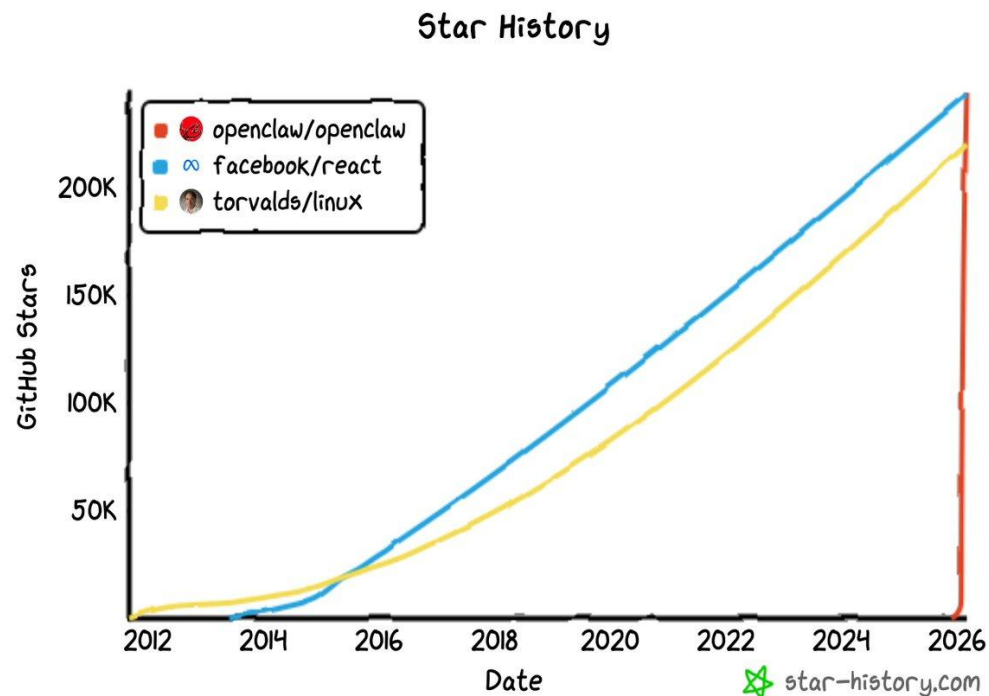


Agentic AI



Agent正在走向真实使用

- 2025年11月 — 奥地利开发者 Peter Steinberger 推上 GitHub
前身为 Clawdbot / Moltbot, 最初只是一个 WhatsApp relay 项目
- 14天 — 获得 190,000+ stars
GitHub 历史上从未有项目达到如此增速
- 84天 — 达到 200,000 stars
超越 Linux (186K stars), 成为第14名
- 2026年3月 — 超越 React, 达到 250,000+ stars
React 维持 #1 地位多年, OpenClaw 用不到4个月完成超越



Home

DMs

Activity

Files

Later

Tools

Admin

Upgrade Plan

Huddles

Drafts & sent

Directories

Starred

Channels

general

new-to-slack

random

rockt

Add channels

Direct messages

dashjim you

Invite people

Apps

Slackbot

OpenClaw-AgentCore

Add apps

MessagesAbout

Today

- 页面交互（点击、填表等）

写作

- 报告、文档、翻译

- 内容总结、改写

定时任务

- 设置 cron 提醒和周期性任务（比如"每天早上9点提醒我开会"）

语音

- 文字转语音（TTS）

消息

- 跨平台发消息（Slack/Telegram/Discord 等，取决于你配置了哪些）

记忆

- 记住你告诉我的事情（跨会话持久化到 S3）

- 搜索历史记忆

设备控制（如果有配对的设备）

- 拍照、录屏、获取位置、发通知

子代理

- 启动独立的 AI 子任务（需要配置 pairing）

简单说：搜索、写东西、跑命令、管文件、定时提醒、浏览网页——基本上一个全能助手能干的事我都能干。限制主要是不能主动发邮件/发推（除非你配置了对应渠道）。

想试什么直接说！

B I U ↺ ↻ ↺ ↻ ↺ ↻ ↺ ↻ ↺ ↻ ↺ ↻

Message OpenClaw-AgentCore

+ Aa 😊 @ 📺 🎤 ↻

➤

OpenClaw 四层架构栈

L4: OpenClaw Gateway

Channel · Memory · Cron · Session · Sandbox · Voice

L3: pi-coding-agent

SessionManager · AuthStorage · Skills · Extensions

L2: pi-agent-core

LLM ↔ Tools

Agent Loop · Steering · Follow-ups

L1: pi-ai — Core LLM Abstraction

streamSimple() / completeSimple() — 统一 streaming 接口

Anthropic · OpenAI · Google · Bedrock · Mistral · Groq · xAI · Ollama · OpenRouter (2000+ 模型)

Channels

WhatsApp
Slack · 20+

Memory

MEMORY.md
Vector+BM25

Cron

Heartbeat
定时任务

Persona

SOUL.md
USER.md

Sandbox

Tool 沙箱
权限控制

Session

JSONL 持久化
Lane Queue

第一步，解决模型调用的复杂性

L1: PI-AI LLM抽象层

streamSimple() / completeSimple() — 统一 streaming 接口，支持 2000+ 模型
Anthropic. OpenAI. Google. Bedrock. Mistral. Groq. XAI. Ollama.
OpenRouter...

第二步，让模型执行任务，开始行动

L2: PI-Agent-core Agent循环引擎

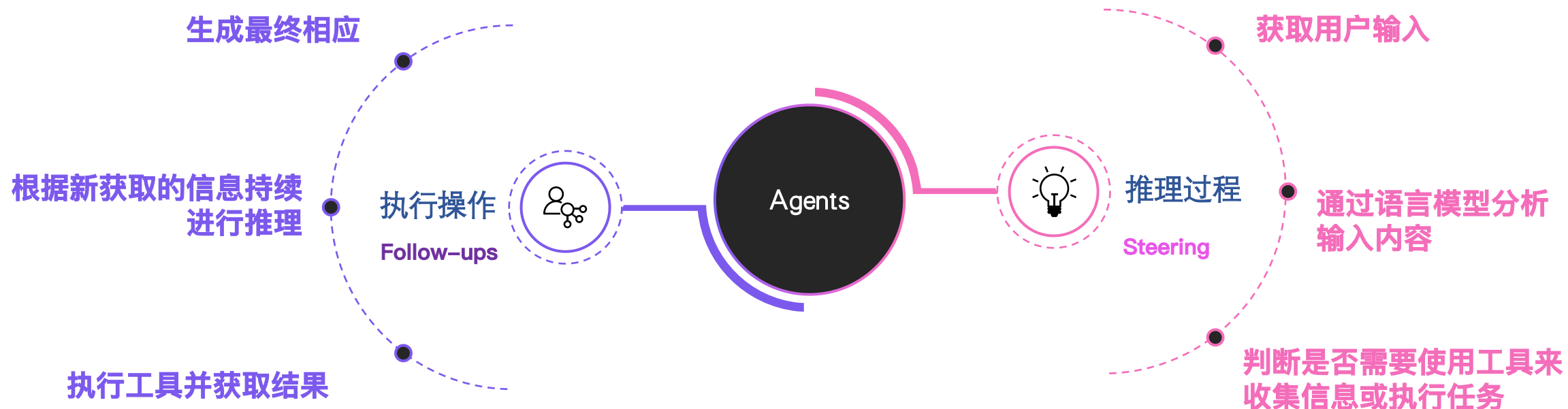
LLM ↔ ToolAgent Loop. Steering. Follow-ups

L1: PI-AI LLM抽象层

streamSimple() / completeSimple() — 统一 streaming 接口，支持 2000+ 模型
Anthropic. OpenAI. Google. Bedrock. Mistral. Groq. XAI. Ollama.
OpenRouter...

Agent Loop

Agent = Loop + Steering + Follow-ups



第三步，让Agent 持续运行

L2: PI- Coding-Agent 完整运行时

Session Manager. AuthStorage. Skills. Extensions

L2: PI-Agent-core Agent循环引擎

LLM ↔ Tool
s

Agent Loop. Steering. Follow-ups

L1: PI-AI LLM抽象层

streamSimple() / completeSimple()— 统一 streaming 接口，支持 2000+ 模型
Anthropic. OpenAI. Google. Bedrock. Mistral. Groq. XAI. Ollama.
OpenRouter...

Session Manager (会话管理)

- 多用户并发
- 多任务隔离
- 上下文持久化
- Agent“记得自己在做什么”

AuthStorage (身份 & 权限)

- API key / credentials 管理
- 用户身份隔离
- 权限控制 (谁能调用什么)

Skills (能力插件)

标准化工具封装

- 发邮件
- 调 Slack
- 查数据库
- 调 AWS

A可扩展的能力平台

Channel Adapters (20+)

WhatsApp · Telegram · Discord · Slack · Signal
iMessage · 飞书 · Google Chat · MS Teams · IRC

Web 搜索

Brave Search API
Tavily (AI优化)
web_fetch 页面抓取

Agent Browser

Playwright 无头浏览器
Chrome Extension Relay
Canvas 渲染引擎

OpenClaw Gateway

Skills (5,700+)

✉ 邮箱 (Himalaya)
📖 Notion · Obsidian
📄 小红书 · Twitter · B站
💻 GitHub · Claude Code

MCP · Nodes

Model Context Protocol
设备节点 · 相机 · 屏幕
跨设备协作

ACP

外部编码工具
Codex · Claude Code

A2A

Agent 间通信
Ed25519 签名

Agent Team

多Agent协作
组织架构

Kanban

任务看板
DAG · MCP

第四步，多个Agent协同工作

L4: OpenClaw Gateway 基础设施层

Channel. Memory. Cron. Session. Sandbox. Voice

L3: PI- Coding-Agent 完整运行时

Session Manager. AuthStorage. Skills. Extensions

L2: PI-Agent-core Agent循环引擎

LLM ↔ Tools

Agent Loop. Steering. Follow-ups

L1: PI-AI LLM抽象层

streamSimple() / completeSimple()— 统一 streaming 接口，支持 2000+ 模型
 Anthropic. OpenAI. Google. Bedrock. Mistral. Groq. XAI. Ollama.
 OpenRouter...

Channels

WhatsupApp,
Slack.feishu..
20+

Memory

MEMORY.md
Vector+BM25

Cron

Heartbeat
定时任务

Persona

SOUL.md
USER.md

Sandbox

Tool 沙箱
权限控制

Session

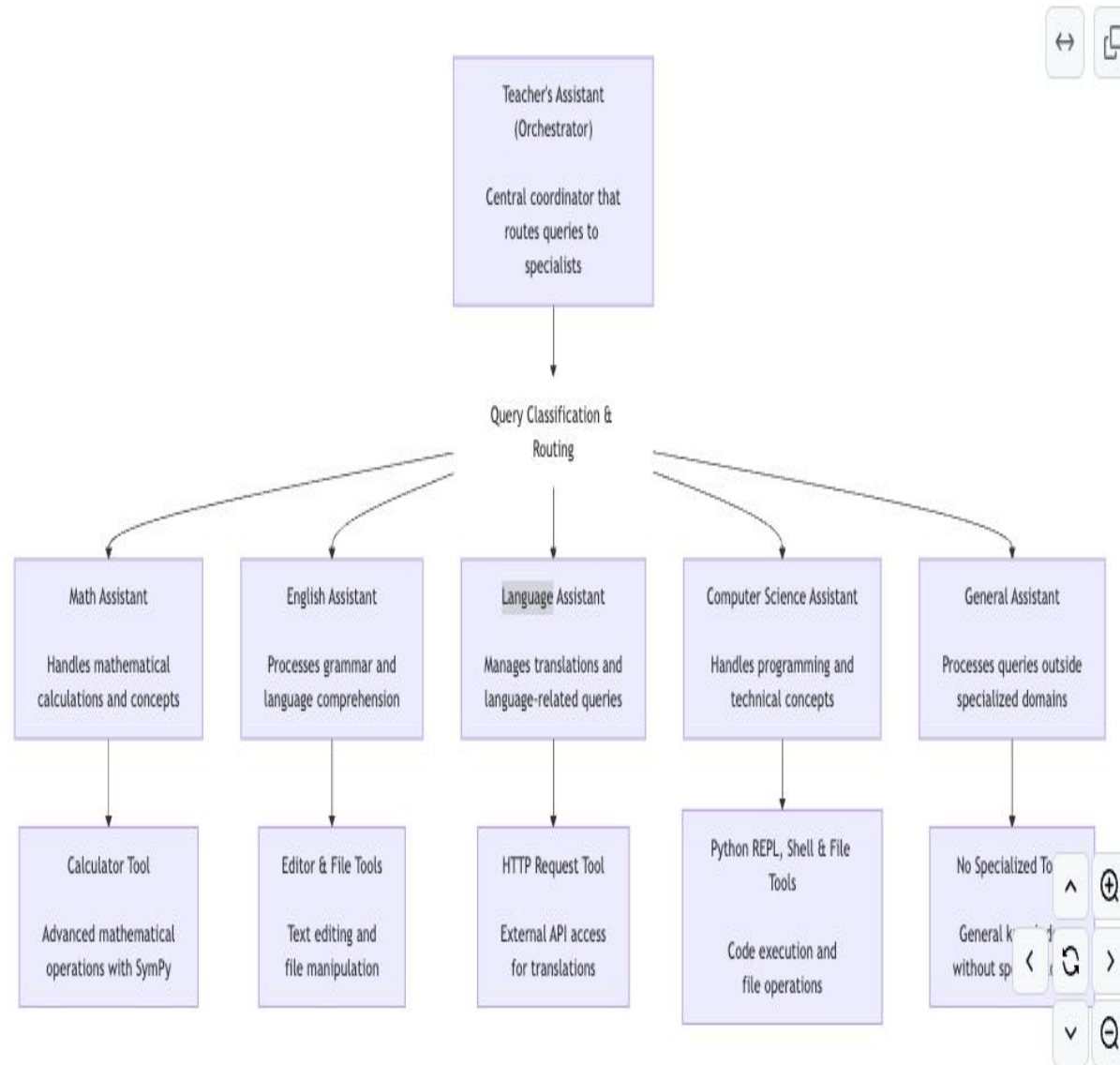
JSONL持久化
Lane Queue

Multi-agent协同模式

Multi-agent模式	描述	相似类比
Agent as Toos (Orchestrator)	Lead-agent 协调多个Sub-agent, Sub-agent作为Lead-agent的工具使用	- Amazon Bedrock Multi-agent - Multi-agent orchestrator - OpenAI agent sdk handoff
Swarm/群体智能	通过共享信息和context, 把任务分给多个Agent并行解决问题	Autogen groupchat
Graph	具有网络拓扑结构的Agent互联网络, Node代表Agent, Edge代表Agent之间的通信	LangGraph
Workflow	通过固定工作流控制Agent信息和执行顺序, 一个Agent的输出是下一个Agent的输入context	LangGraph

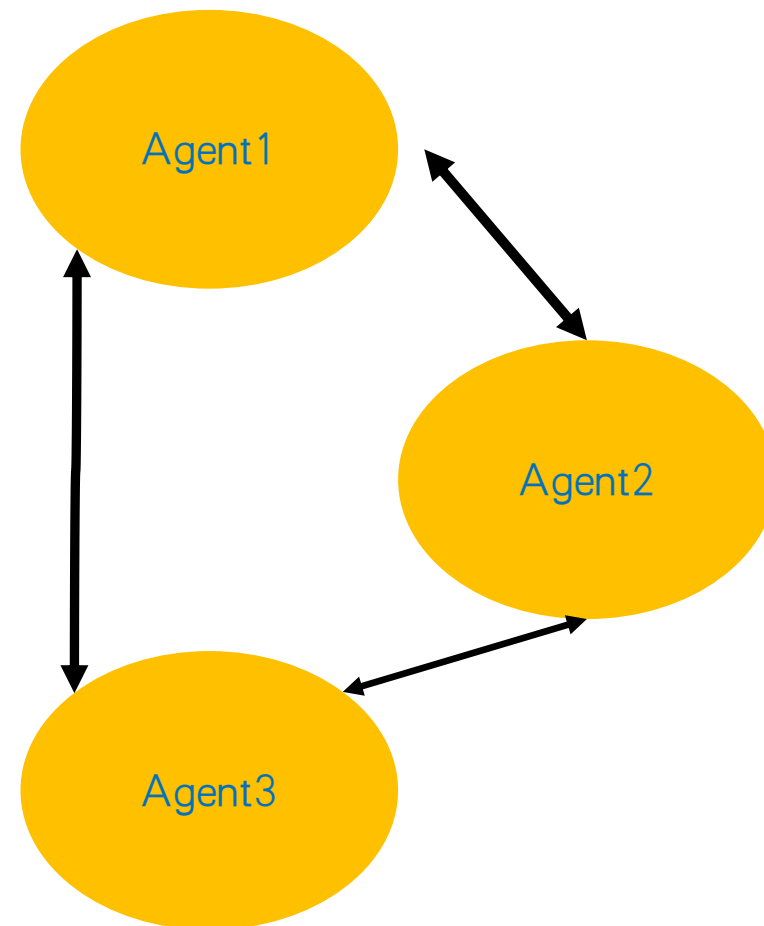
Orchestrator 模式 (Agents as tools)

- 分而治之：每个专家Agent都有明确的责任领域，使系统更容易理解和维护
- 中央集权，层级委派：协调者决定调用哪个专家Agent，形成明确的指挥链条
- 模块化架构：Agent专家可以独立优化，添加、删除或修改，甚至复用到其他的系统中



Swarm 群体智能

- 去中心化：每个Agent相对独立且并行执行任务，没有上级指挥。单个Agent 相对简单，通过群体协作达成更高难度任务
- 上下文信息共享：每个Agent都需要看到其他Agent的输入输出信息
- 多种协作方式： Collaborative, Competitive, Hybrid



示例：使用swarm tool动态创建multi agents

Collaborative 模式下，每个Agent会在其他Agent的基础上进行完善

```
agent = Agent(tools=[swarm])
result = agent.tool.swarm(
    task="AI对人类发展的影响是什么？",
    system_prompt="你是一个善于思辨的哲学家",
    swarm_size=4,
    coordination_pattern="collaborative",
)
```

```
{'text': '🤖 Agent agent_1: Response: # AI对人类发展的影响: 综合视角\n\n分析前几位专家的观点, 我看到了我们对AI影响的多层次认识。我想在此基\n{'text': '🤖 Agent agent_1: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=17.135s, total_time=17.135s\n{'text': '🤖 Agent agent_2: Response: # AI对人类发展的影响: 整合与扩展视角\n\n我想基于前一阶段的讨论, 对AI对人类发展影响的分析进行整合与扩\n{'text': '🤖 Agent agent_2: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=23.725s, total_time=23.725s\n{'text': '🤖 Agent agent_3: Response: # AI对人类发展的影响: 整合视角\n\n基于之前的讨论, 我想从以下几个角度进一步整合与深化AI对人类发展影响\n{'text': '🤖 Agent agent_3: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=57.481s, total_time=57.481s\n{'text': '🤖 Agent agent_4: Response: # AI对人类发展的综合影响分析\n\n我将整合并深化前一阶段讨论的洞见, 提供对AI影响人类发展的更全面思考。\n{'text': '🤖 Agent agent_4: Metrics: Event Loop Metrics Summary:\n| Cycles: total=1, avg_time=21.360s, total_time=21.360s\n{'text': '\n🌟 Collective Knowledge:\n\n { "id": "3f0dba92-e10c-4005-8d83-9ee826bf7644", "agent_id": "agent_3"
```

Competitive 模式下，每个Agent会在其他Agent的基础上补充其他的视角

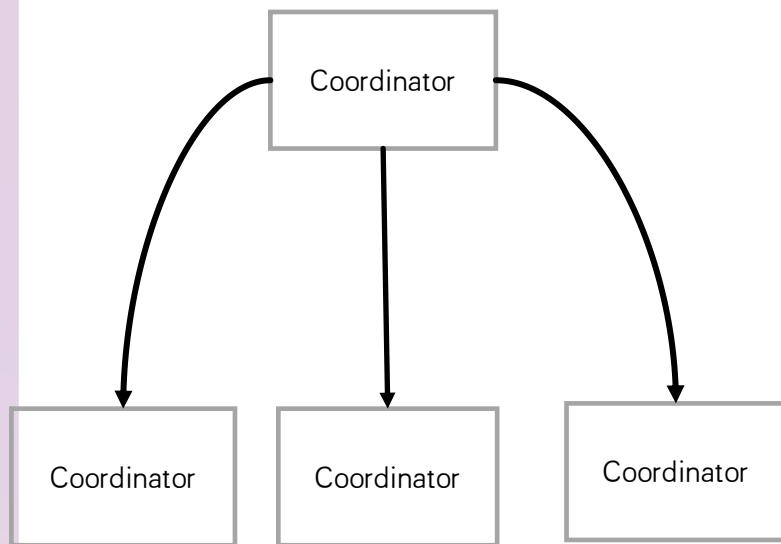
```
result = agent.tool.swarm(
    task="AI对人类发展的影响是什么？",
    system_prompt="你是一个善于思辨的哲学家",
    swarm_size=4,
    coordination_pattern="competitive",
)
```

```
角\n\n基于前一阶段的讨论, 我要从长期共生演化角度分析AI对人类发展的影响, 这是一个之前未被充分探讨的维度:\n\n## 人类认知与AI系统的协同进化\n\n人类认\n- Cycles: total=1, avg_time=19.229s, total_time=19.229s\n| Tokens: in=7641, out=539, total=8180\n| Bedrock Latency: 18908ms\n分配视角\n\n在前一轮讨论中, 我们看到了从生态系统、经济效益、认知重构和社会结构等多个角度对AI影响的分析。我想提供一个独特的补充视角: AI如何重塑全球资源\n- Cycles: total=1, avg_time=50.601s, total_time=50.601s\n| Tokens: in=7641, out=644, total=8285\n| Bedrock Latency: 16762ms\n重塑视角\n\n根据以往的分析, AI对人类发展的影响已从技术、经济、社会结构等多个维度进行了探讨, 但对伦理与价值观的重塑尚未深入。从这一独特视角出发:\n\n# \n- Cycles: total=1, avg_time=24.212s, total_time=24.212s\n| Tokens: in=7641, out=564, total=8205\n| Bedrock Latency: 23884ms\n转型视角\n\n我将从伦理与价值观转型的独特视角探讨AI对人类发展的影响, 这是前面分析中尚未充分展开的领域:\n\n## 价值观体系的重构\n\nAI系统内嵌的价值取\n- Cycles: total=1, avg_time=18.695s, total_time=18.695s\n| Tokens: in=7641, out=483, total=8124\n| Bedrock Latency: 18386ms
```

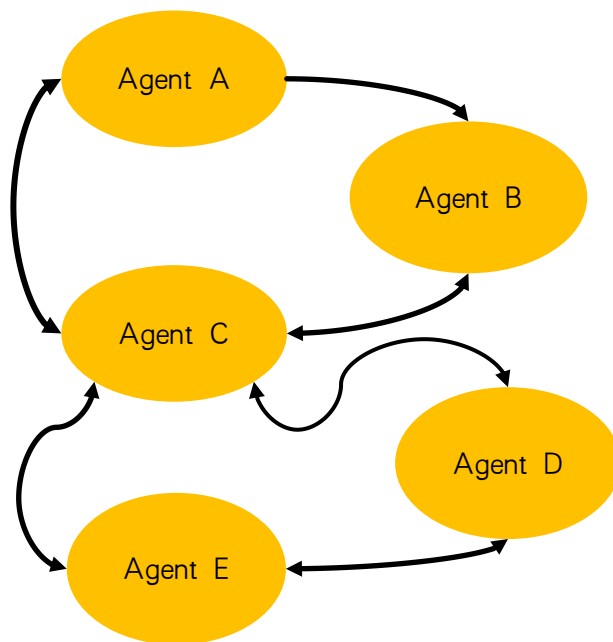

Agent Graph

- 利用Agent as Tools 机制组成更复杂的Agent 网络
- 支持拓扑类型： star, mesh, hierarchical

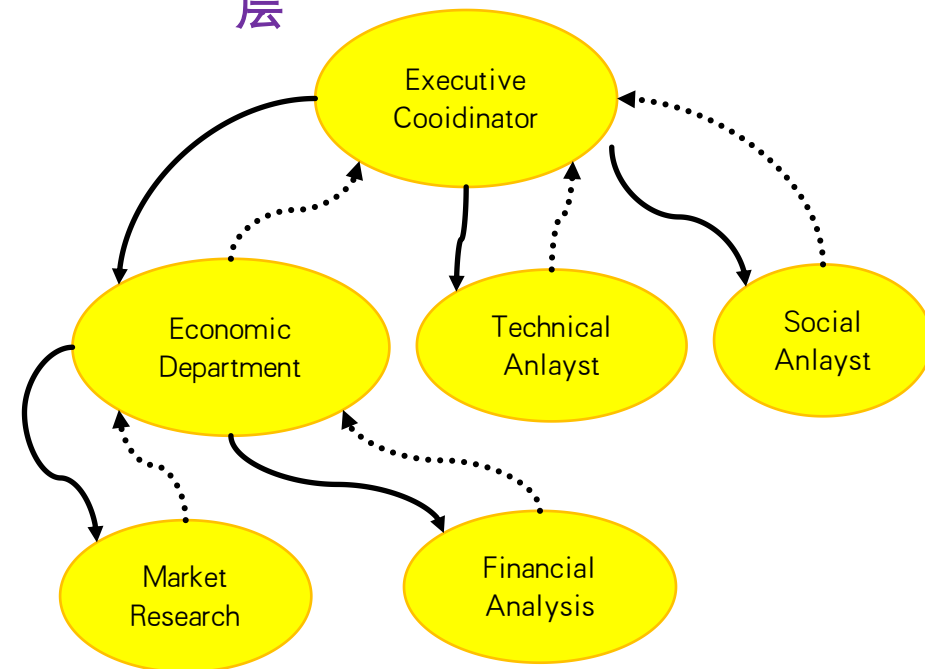
**Star/星
型**



Mesh/网状



**Hierarchical/分
层**





示例：使用Agent graph创建multi agents

方法1:使用自然语言动态创建Agent graph

```
from strands import Agent
from strands_tools import agent_graph
# Initialize Agent with agent_graph
agent = Agent(tools=[agent_graph])

# Create a network through natural language
agent("Create a research team with a coordinator, data analyst, and domain expert in a star topology")

# Send a task to the team
agent("Ask the research team to analyze emerging technology adoption in healthcare")

# Check status and manage the network
agent("Show me the status of the research team")
```

```
agent("Show me the status of the research team")
```

```
# Check status and manage the network
```

Code sample

https://github.com/strands-agents/tools/blob/main/src/strands_tools/agent_graph.py

这个工具可以让agent自行创建Sub-Agent并组成网络

```
TOOL_SPEC = {
    "name": "agent_graph",
    "description": """"Create and manage graphs of agents with diff
```

Key Features:

1. Multiple topology support (star, mesh, hierarchical)
2. Dynamic message routing
3. Parallel agent execution
4. Real-time status monitoring
5. Flexible agent configuration

Example Usage:

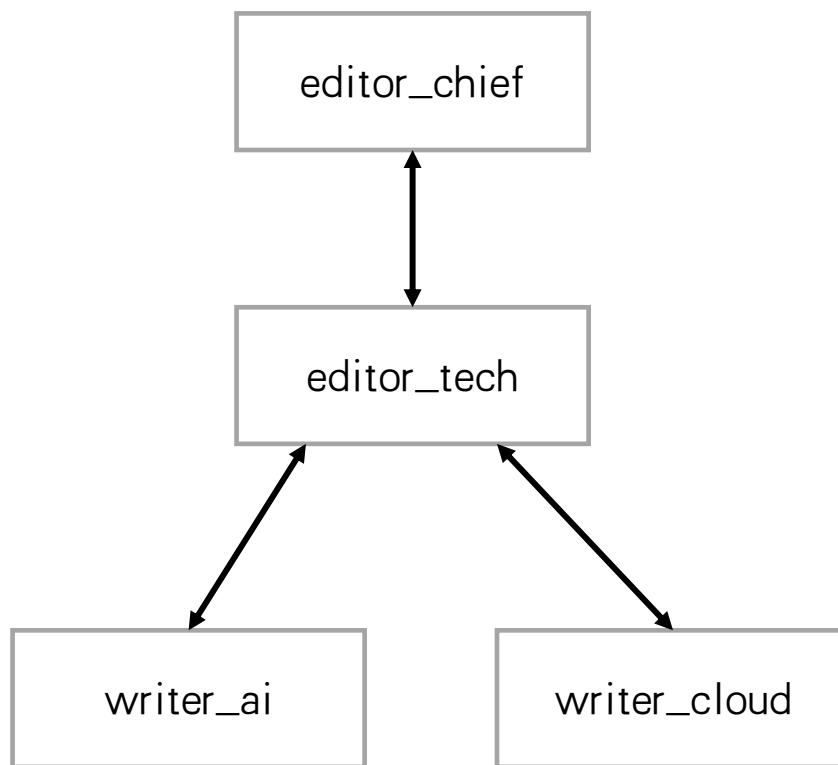
1. Create a new agent graph:

```
{
    "action": "create",
    "graph_id": "analysis_graph",
    "topology": {
        "type": "star",
        "nodes": [
            {
                "id": "central",
                "role": "coordinator",
                "system_prompt": "You are the central coordinator."
            },
            {
                "id": "agent1",
                "role": "analyzer",
                "system_prompt": "You are a data analyzer."
            }
        ],
        "edges": [
            {"from": "central", "to": "agent1"}
        ]
    }
}
```

示例：使用Agent graph创建multi agents

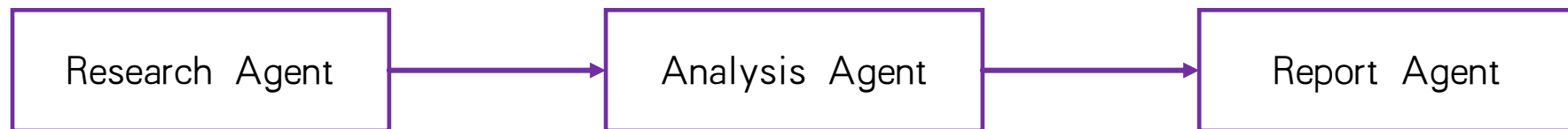
方法2: 代码指定 graph

```
# Initialize Agent with agent_graph
agent = Agent(tools=[agent_graph])
# Example: Create a content production team
agent.tool.agent_graph(
    action="create",
    graph_id="content_team",
    topology={
        "type": "hierarchical",
        "nodes": [
            {"id": "editor_chief", "role": "executive",
             "system_prompt": "You provide strategic direction and final approval."},
            {"id": "editor_tech", "role": "manager",
             "system_prompt": "You manage technical content creation."},
            {"id": "writer_ai", "role": "specialist",
             "system_prompt": "You write about AI technologies."},
            {"id": "writer_cloud", "role": "specialist",
             "system_prompt": "You write about cloud computing."}
        ],
        "edges": [
            {"from": "editor_chief", "to": "editor_tech"},
            {"from": "editor_tech", "to": "writer_ai"},
            {"from": "editor_tech", "to": "writer_cloud"},
            {"from": "writer_ai", "to": "editor_tech"},
            {"from": "writer_cloud", "to": "editor_tech"},
            {"from": "editor_tech", "to": "editor_chief"}
        ]
    }
)
```



Workflow

- 执行多步骤，有顺序和依赖条件的复杂任务
- 并行执行分支，支持指定执行优先级（1-5）
- workflow的状态生成Json 文件，持久化保存。可以暂停，恢复



★AstronClaw 示例：使用Agent workflow 创建Agentic workflow

方法1:使用自然语言动态创建workflow

自动创建workflow.json

```
# Create an agent with these tools
agent_translation = Agent(tools=[workflow], callback_handler=custom_ca

agent_translation(f"""
Create a translation workflow with the fol
1. A AI tech translator translate news con
2. Review translation and provide feedback
3. Refine the translation based on feedbac
4. Generate 3~5 insights (depends on task
""")

text="""# AWS intros Strands Agents SDK

Many developers find agentic AI difficult,
Esther Shittu
By..."""

agent_translation(f"translate {text}")
```

```
{
  "workflow_id": "news_translation_workflow",
  "created_at": "2025-05-20T05:50:28.096395+00:00",
  "status": "completed",
```

```
  agent_translation(f"translate {text}")
```

✓ 1m 52.9s

虽然集成角度相对间接，但AWS使用独立生成式AI供应商Anthropic的模型复杂协议（MCP），"这打开了许多生态系统的大门，"Stephens说。"MCP是连接智能体工具的事实标准。"

Strands Agents可以在任何地方运行，并支持任何具有推理和工具使用能力的模型，包括Amazon Bedrock和Ollama中的模型。作为开源系统，Strands Agents包括来自社区其他成员的贡献，包括Accenture、Anth

与此同时，OpenAI周五推出了Codex，一个基于云的软件工程智能体，可以并行处理编写功能、回答关于代码库的问题和修复错误等任务。

它由codex-1提供支持，这是OpenAI的o3 LLM的一个版本，针对软件工程进行了优化，并通过强化学习进行了训练，OpenAI表示。

新智能体现在将向ChatGPT Pro、Enterprise和Team用户推出。Plus和Edu用户也将很快能够使用它。

Esther Shittu是Informa TechTarget的新闻作家和播客主持人，报道AI软件和系统。

...

Key Insights Generated:

...

关键见解

1. ****AI智能体开发简化的趋势****: AWS通过Strands Agents展示了行业正在努力简化智能体AI的开发过程，让普通开发者无需深厚专业知识也能部署AI智能体。这表明AI工具正在从专家领域向主流开发者社区过渡。
2. ****SDLC整合是云服务商的新战略方向****: AWS将Strands Agents与软件开发生命周期(SDLC)整合的努力，表明主要云服务提供商正扩展其价值主张，从仅提供基础设施向成为开发流程的核心部分转变。
3. ****模型复杂协议(MCP)成为行业标准****: 文章指出MCP成为连接智能体工具的"事实标准"，这凸显了在快速发展的AI领域中，开放标准和互操作性变得越来越重要，可能会影响未来AI开发的格局。
4. ****开源协作成为AI发展关键动力****: Strands Agents作为开源系统接受了包括Accenture、Anthropic、Meta和PwC在内的多家公司的贡献，展示了开源合作在推动AI技术发展中的关键作用。
5. ****AI工具竞争加剧****: AWS推出Strands Agents的同时，OpenAI也发布了Codex，显示主要AI公司正在智能体市场展开直接竞争，这可能会加速技术创新并使开发者受益。

...

The entire workflow was completed successfully. The news article about AWS introducing Strands Agents SDK was translated into Chinese, reviewed for quality, refined

```
"translate_to_chinese": {
  "status": "completed",
  "result": [],
  "completed_at": "2025-05-20T05:51:52.689112+00:00"
```




示例：使用Agent workflow 创建Agentic workflow

方法2: 使用代码定义workflow

```
# Connect to an MCP server using stdio transport
stdio_mcp_client = MCPClient(lambda: stdio_client,
                               StdioServerParameters(command="npx", args=
))

streamable_http_mcp_client = MCPClient(lambda:
url="https://hapi.us-east-1.amazonaws.com/mcp",
headers={"Authorization": "Bearer "})

# Create an agent with MCP tools
with streamable_http_mcp_client:
    with stdio_mcp_client:
        # Get the tools from the MCP server
        tools = stdio_mcp_client.list_tools_sync()
        for tool in tools:
            print(tool.tool_name, tool.tool_description)

# Sequential workflow processing
def process_workflow(topic):
    # Create specialized agents
    researcher = Agent(system_prompt="You are a research assistant. Use the provided tools to gather information.",
                       tools=tools, capabilities=capabilities)
    analyst = Agent(system_prompt="You are an analyst. Synthesize the research findings into a coherent analysis.",
                   tools=tools, capabilities=capabilities)
    writer = Agent(system_prompt="You are a writer. Format the analysis into a professional report.",
                  tools=tools, capabilities=capabilities)

    # Step 1: Research
    research_results = researcher(f"Research the latest trends in AI agent technology.")

    # Step 2: Analysis
    analysis = analyst(f"Analyze these research results: {research_results}")

    # Step 3: Report writing
    final_report = writer(f"Create a comprehensive report on AI agent technology trends based on the analysis.")

    print(f"\n----final-----:\n{final_report}")

process_workflow("AI Agent technology trends")

----final-----:
# EXECUTIVE REPORT: AI AGENT TECHNOLOGY TRENDS
## Strategic Implications for Enterprise Decision Makers

### EXECUTIVE SUMMARY
AI agent technology stands at an inflection point of extraordinary growth and business impact. Recently identified by Gartner as the #1 strategic technology trend for 2024, this emerging paradigm is reshaping the digital landscape.

### MARKET TRAJECTORY
Our analysis confirms the AI agent market is entering a phase of explosive growth:

* **Current market valuation**: $5.1 billion (2024)
* **Projected market size**: $47.1 billion by 2030
* **Compound Annual Growth Rate (CAGR)**: 44.8%
* **Enterprise adoption**: 33% of enterprise software applications will include agentic AI by 2028

This growth trajectory positions AI agents among the fastest-adopted emerging technologies in recent history, signaling a fundamental shift in how organizations will approach digital transformation.

### KEY TECHNOLOGY DEVELOPMENTS

**1. Enterprise-Grade Infrastructure**
Microsoft's Azure AI Foundry Agent Service has reached general availability, providing organizations with a fully managed platform for building, deploying, and scaling AI agents.

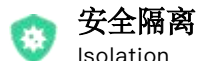
**2. Standardization Initiatives**
The Model Context Protocol (MCP) is emerging as the critical standard for AI agent interoperability. Originally developed by Anthropic, MCP now has widespread industry adoption, enabling seamless communication between different AI systems.

**3. Agent-Ready Web Architecture**
Microsoft's introduction of NLWeb—described as "HTML for the agentic web"—represents a significant advancement in making digital content and services accessible to AI agents, paving the way for more integrated and useful agent-based applications.
```

当这些 Agent 进入企业环境时，问题才真正开始

从 "玩具" 到 "生产力工具"，不同类型的企业都遇到了各自的阻碍

痛点



安全隔离
Isolation



企业治理
Security



可观测性
Observability



成本压力
Cost



金融科技公司

处理 KYC 等敏感数据

安全隔离

“OpenClaw 权利太大，客户 A 的数据绝对不能被客户 B 看到。Docker 容器虽然轻量，但共享内核让我不放心，一旦逃逸就是重大 **合规事故**。”



大型企业集团

200+ Agent 服务多部门

企业治理

“OpenClaw Skills 默认继承 Agent 全量权限，市场部的人能调财务 API；1,184 个恶意插件在 ClawHub 上，我没法统一管控 **鉴权** 谁装了什么”



AI 产品公司

日均 10K+ 次执行

可观测性

“OpenClaw 没有系统化的可观测体系，我不知道谁的 Agent 调了什么工具、访问了什么数据，出了安全事件无法追溯，也无法评估 Agent 整体质量我希望能 **日志能追溯**。”



快速扩张创业公司

预算有限但需 7x24 服务

成本失控

“开了 20 台 EC2 跑 7x24，但 OpenClaw 大部分的时间其实是在等 **LLM 响应**，不在处理任务。计算资源在空转，照样全价收费，**成本较高**不经济。”



OpenClaw 在无治理下的安全风险

5大关键威胁	严重性	发生了什么（真实案例 / 风险描述）
 提示词注入	Critical	接管 OpenClaw 控制权 Zenity Labs 研究人员通过文章注入恶意提示词成功接管；Archestra.ai CEO 通过一封邮件让 OpenClaw 泄露了 SSH 私钥。
 恶意 Skills 供应链	Critical	1,184 个恶意插件被发现 ClawHavoc 事件数周内激增 142% ；Trend Micro 发现 39 个恶意 Skills 分发 macOS 信息窃取器。
 源代码漏洞	High	82 个已知 CVE (截至 2026 年 3 月) 包含 12 个超危 和 21 个高危漏洞，涉及远程代码执行 (RCE) 等核心安全问题。
 凭据泄露	High	220,000+ 实例暴露公网 SecurityScorecard 数据：默认配置下凭据明文存储，一旦暴露即导致全量密钥泄露。
 企业数据泄露	High	无最小化权限约束 36.8% 社区插件存在安全问题；Agent 默认继承全量权限，极易导致敏感数据越权访问。

(*) OWASP 2026 发布 Agentic Applications Top 10 安全威胁；OpenClaw 面临双重风险：开源软件风险 + AI Agent 风险

OpenClaw 风险

- 能访问什么
- 被授权自主执行什么

当 Agent 同时拥有 代码执行 + 工具调用 + 数据访问 三种能力时，风险最高 — 而 OpenClaw 恰好三者兼具。

1 不对外暴露OpenClaw

- 除非必须，否则应使用localhost
- 对外暴露时通过防火墙严格控制访问
- 对外暴露时使用添加WAF保护及Ratelimit限制
- 对外暴露时记录访问日志并监控异常

2 定期扫描并修复漏洞

- 扫描Host的漏洞并及时安装更新补丁
- 定期扫描OpenClaw的漏洞并及时更新版本
- 不使用非官方来源的安装包或升级包
- 对外暴露时记录访问日志并监控异常

3 使用隔离的环境运行

- 将OpenClaw部署在独立的虚拟机中
- 通过防火墙控制OpenClaw可访问网络及域名
- 使用OpenClaw的[沙箱](#)功能运行Tool

4 谨慎安装Skills

- 仅使用可信来源的Skills
- 检查Skills的安全扫描结果
- 企业应维护自己的Skills仓库并谨慎审查Skills再入库
- 对于可疑Skills，在沙箱中运行并分析其行为

5 谨慎处理外部数据

- 尽可能只访问内部系统及数据
- 若需访问外部数据，应部署防火墙阻止访问恶意站点

6 仅赋予最小权限

- 使用非Root用户运行
- 通过tools.allow/deny来仅允许调用必要的tool
- 仅授予必要的外部系统（如邮件系统）权限，例如只读
- 对于危险操作（如删除）在OpenClaw外部实现人工确认机制

7 检查不安全的配置

- 从[最小配置基线](#)开始，按需修改配置
- 修改后执行openclaw security audit命令检查不安全的配置
- 对配置文件进行完整性监控

8 安全管理密钥

- 为OpenClaw所使用的凭据添加IP限制，降低凭据丢失的影响
- 使用Secrets Manager安全存储密钥，OpenClaw按需加载
- 定期轮转密钥
- 保护“.openclaw”目录，仅允许openclaw用户读写

9 使用身份委托避免越权访问

- OpenClaw访问外部系统时应传播用户身份，使用最终用户的身份来访问外部系统
- 若OpenClaw被授予固定身份访问外部系统，应为OpenClaw的访问添加权限控制，避免通过其完成权限提升

10 部署运行时安全监控

- 部署终端安全工具（如GuardDuty）监控Host的安全事件
- 对异常行为及时告警
- 添加安全事件的自动响应，例如自动隔离Host的网络

11 维护Agent资产清单并监控未授权的安装

- 扫描主机是否有监听OpenClaw常用端口，如18789
- 扫描主机是否有.openclaw目录
- 及时记录合规的Agent，清理未经审批的实例

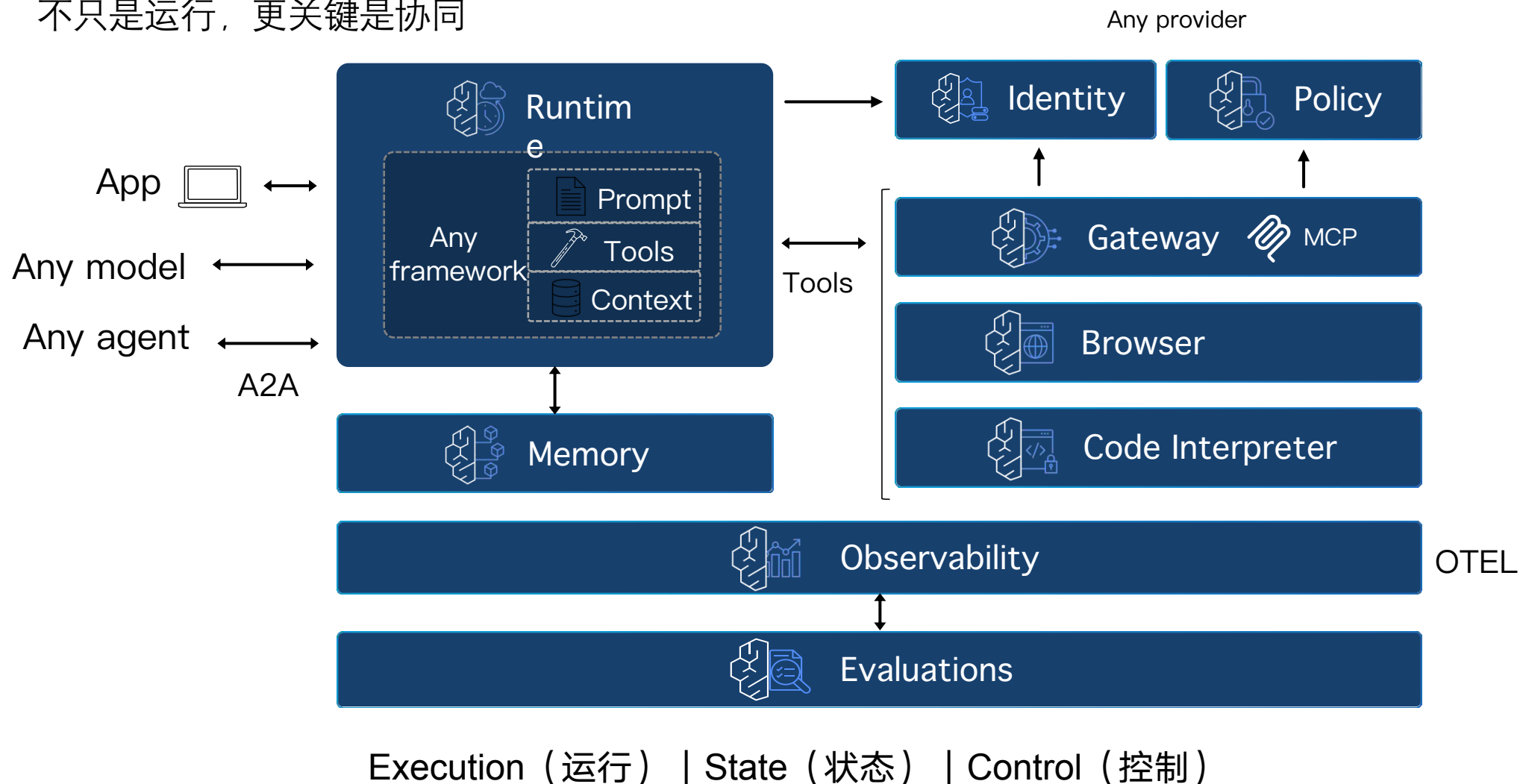
12 将 AI Agent 纳入信息安全治理体系

- 制定 AI Agent 相关安全策略、标准和流程
- 将 AI Agent 常见风险纳入安全意识培训
- 对不符合公司安全策略的AI网站或工具使用防火墙和终端安全工具进行封禁



选择云上构建，让Agent真正进入生产环境

不只是运行，更关键是协同



维度1: 安全隔离同时兼顾按需伸缩

Architecture View

→ 每个用户一个独立沙箱, Serverless 自动伸缩

Firecracker microVM



A

A的龙虾会话

独立 CPU / 内存 / 文件系统

完全隔离

Firecracker microVM



B

B的龙虾会话

独立 CPU / 内存 / 文件系统

完全隔离



按需自动伸缩

Serverless

E2E冷启动: 5s

最长运行: 8 小时

会话结束: 自动清理, 零残留

隔离技术: Firecracker

用户隔离



会话隔离

传统容器方案 (路径3)

- ✗ **共享内核:** 仅依赖 Namespace/Cgroup, 攻击面大
- ✗ **软隔离:** 存在容器逃逸风险(CVE-2024-21626)
- ⚠ **数据残留:** 销毁不彻底, 本地卷风险

AgentCore

Firecracker microVM (路径4)

- ✓ **硬件级虚拟化:** 基于 KVM, 独占 CPU/Mem
- ✓ **强隔离:** 攻击者无法跨越 VM 边界
- ✓ **阅后即焚:** 会话结束毫秒级销毁



Demo: OpenClaw 企业级隔离

两只 OpenClaw 无法查看对方的密钥 01:01

<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>



维度2: 企业治理 — 快速实现预设的企业规则

自建

数十周



Policy Engine

控制 Agent/用户 调用的工具与参数

2-4 周开发 + 维护



Identity 集成

对接 IdP (Okta/AD) 实现 SSO

2-3 周开发 + 测试



API Gateway

拦截工具调用, 限流 + 认证 + MCP

3-6 周开发



审计系统

全链路日志采集 + 存储 + 查询

2-3 周开发



凭据管理

集成 Secrets Manager + 轮换

1-2 周开发

合计 5 个子系统

1-2 名工程师全职投入

VS



全托管 (AgentCore)

数小时



内置 Policy

Cedar 声明式规则, 无需编码

No Code



原生 Identity

无缝集成 IdP, SSO 开箱即用

Config



托管 Gateway

全链路拦截 + 协议自动转换

Built-in



原生可观测性

全链路 Trace 开箱即用

Native



安全凭据

Secrets Manager 集成

Secure

全部内置, 天级配置

非月级开发

自建方案



2.5 - 4.5 个月全职工程投入



AgentCore 方案



小时级配置, 开箱即用

基于身份的精细化治理 vs 默认全量权限：如何阻止内部数据泄露



大型企业环境

200+ Agent 服务不同部门



调用者：

市场部 (Marketing) 员工的龙虾 Agent



风险操作：

调用 **财务系统 API** 获取预算



核心隐患：

Agent 默认继承全量权限，无边界

// AgentCore Cedar 策略示例

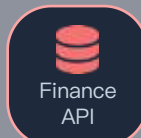
```
forbid ( principal in Group::"Marketing", action ==  
  Action::"GetBudget", resource ==  
  Tool::"FinanceAPI" );
```

不做企业治理

隐式信任



Market
Claw



Finance
API

调用成功：数据泄露

事后审计才发现
无法阻止实时访问

做企业治理 (AgentCore Policy)

防护流程



1. Identity 身份识别

系统识别为 "Marketing Claw User"



2. Policy 规则匹配

策略：市场部 禁止访问财务工具



3. Gateway 实时拦截

请求阻断，无法访问



4. Observability 审计

自动记录拦截日志，触发告警



Demo: OpenClaw 企业权限管控

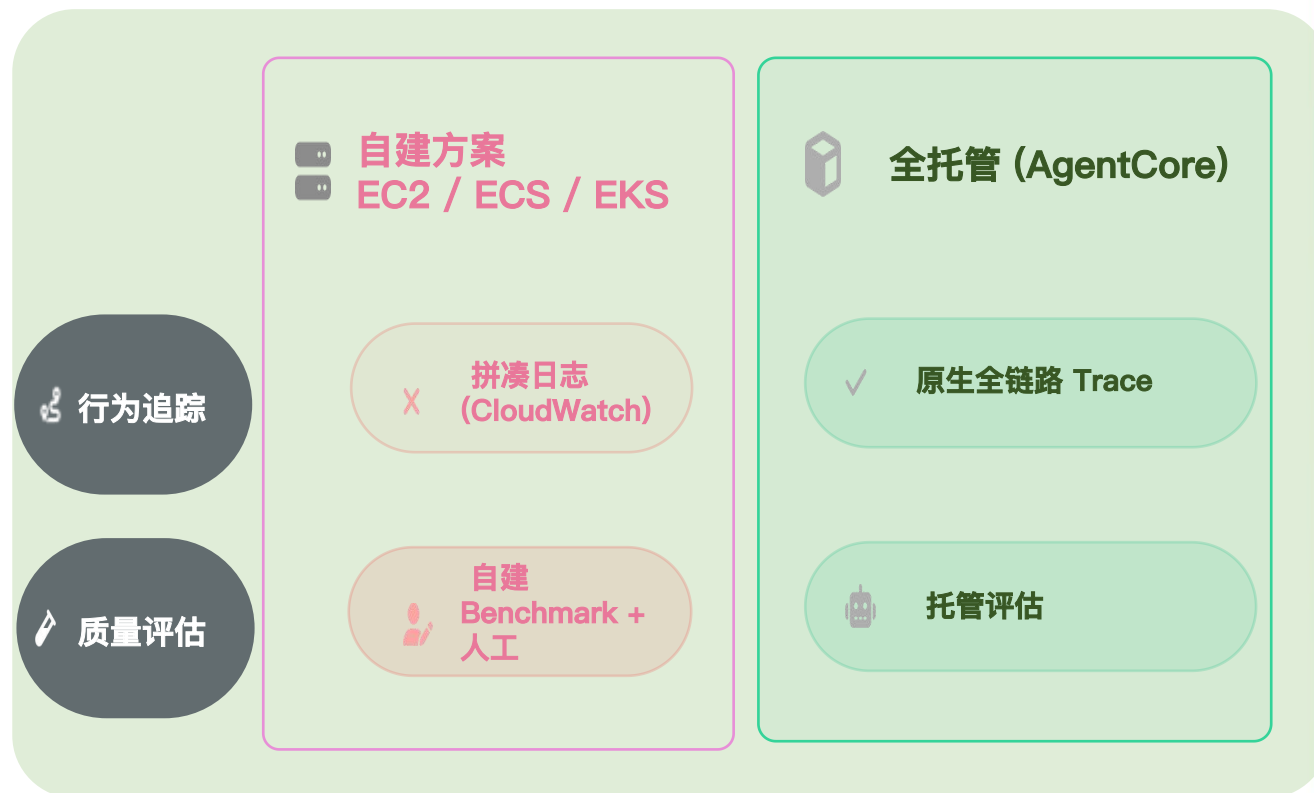
通过 AgentCore Policy 实现 OpenClaw 企业级权限管控

<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>

维度3: 可观测性 — 随时监控您龙虾平台的状态

超过 53% 企业用户在短短一个周末内就给 OpenClaw 授予了特权访问权限。问题不只是“装得快”，而是企业通常**缺乏配套的治理、审计与运行时监控能力**，难以验证 Agent 是否始终在安全、合规、可控的边界内运行。

[Noma Security 数据](#)



<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>

维度4: 成本压力 — 一个龙虾还好，成千上万个呢

基础设施节省分析

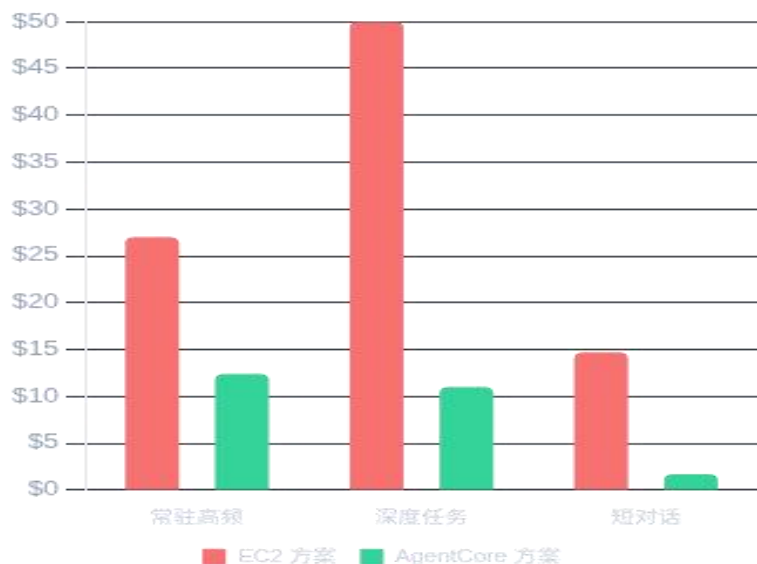
"场景越碎片化、闲置时间越长，按需付费的优势越明显。"

场景 1 (高频)
54%

场景 2 (深度)
78%

场景 3 (短对话)
88%

单人月度基础设施成本 (\$)



Up to 88%
(基于上述假设)

场景 1: 7x 24 常驻助手，高频使用

假设: 100次/天, 30天, t4g.medium常驻 vs Agent按需

节省
54%

项目 (10/100/500人)	10人	100人	500人
EC2 (t4g.medium \$25+EBS \$2) 7x24 常驻 (730h/月)	\$270	\$2,700	\$13,500
AgentCore Runtime CPU \$6.7 + Memory \$5.7 (按需)	\$124	\$1,240	\$6,200
基础设施节省	54%	54%	54%

场景 2: 工作日深度任务，长时间 session

假设: 8h session (活跃4h), 22天, t4g.large vs Agent按需

节省
78%

项目 (10/100/500人)	10人	100人	500人
EC2 (t4g.large \$48+EBS \$2) 支撑长时间复杂任务	\$500	\$5,000	\$25,000
AgentCore Runtime CPU \$7.9 + Memory \$3.3	\$110	\$1,100	\$5,500
基础设施节省	78%	78%	78%

场景 3: 个人助手，工作时间短对话

假设: 100次/天 (60s session), 22天, t4g.small常驻 vs Agent按需

节省 88%

项目 (10/100/500人)	10人	100人	500人
EC2 (t4g.small \$12+EBS \$2.4) 7x24 常驻	\$147	\$1,466	\$7,330
AgentCore Runtime CPU \$1.0/人 + Memory \$0.7/人	~\$17	~\$170	~\$850
基础设施节省	88%	88%	88%

注: 比较不包含模型成本比较，取决于选择的模型；辅助服务 (S3/DTO/etc)，两方案均需要。

云上托管是高性价比和强安全性的优选择

- 隔离性
- 权限治理与安全性
- 可观测性
- 基础设施成本
- 运维难度
- 持久化存储

Virtual Machine

VM 级隔离

较强

全部自建

较贵

较高

✓原生支持

Container

容器级

较强

全部自建

较贵

较高

✓原生支持

Cloud Native Agent Service

最佳选择

✓ microVM 硬件级

✓ 最强

✓ 原生支持的 Agent 全链路观测性

✓ 最便宜, I/O 等待不收费

✓ 较低

⌚ 自建 S3 持久化存储方案, 原生持久化存储 (Coming)

Demo: OpenClaw 端到端体



用户登录开始使用 OpenClaw on AgentCore 方
案

01:06

<https://github.com/aws-samples/sample-OpenClaw-on-AWS-with-Bedrock>

Builder Center

Home

Learn

Topics

Build

Capabilities

Toolbox

Workshops

Connect

Events

Spaces

Community

Heroes

Community Builders

User Groups

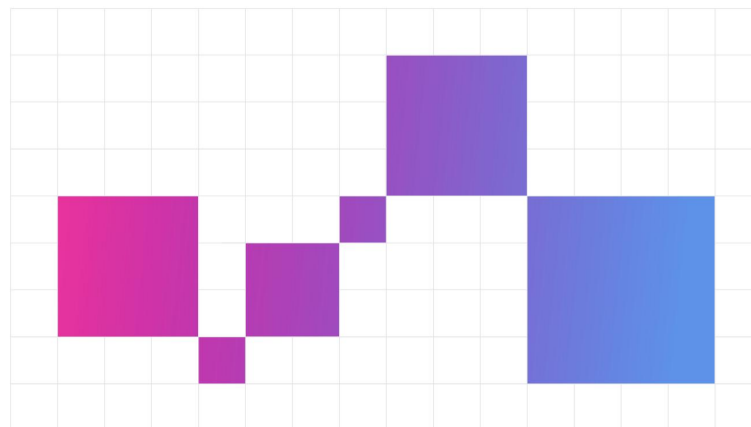
Cloud Clubs

Wishlist

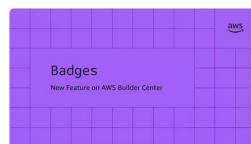
Your ideas. Your community. Your AWS.

Connect with builders who understand your journey. Share solutions, influence AWS product development, and access useful content that accelerates your growth. Your community starts here.

Join the community



Spotlight



Introducing Badges on AWS Builder Center



Elastic Container Service (ECS): My default choice f...



Meet the newest AWS Heroes!



Hov Hall

Spaces



10000 Aldeas...

ai



Join

The official space for the 10,000 Aldeas Competition, starting December 5! This i...

3177

Public



Techkriti 2026 IIT...



Join



了解更多

THANKS

